

Đặc tả: Đăng ký Workshop (Workshop Registration)

1. Mô tả

Tính năng cho phép sinh viên chọn một workshop từ danh sách và thực hiện đăng ký “giữ chỗ” (hold slot). Đối với workshop miễn phí, đăng ký sẽ thành công ngay lập tức và sinh ra mã QR. Đối với workshop có phí, hệ thống sẽ giữ chỗ tạm thời và chuyển sinh viên sang luồng thanh toán. Tính năng này phải giải quyết bài toán “tranh chấp chỗ ngồi” (Concurrency) khi có hàng ngàn sinh viên thao tác cùng lúc.

2. Luồng chính (Happy Path)

Bối cảnh (Context): Sinh viên đã đăng nhập thành công vào hệ thống, đang ở trang chi tiết của một workshop và quyết định bấm nút đăng ký. Lượng truy cập lúc này có thể đang ở mức đỉnh điểm (Peak load).

Các bước xử lý chi tiết (Flow):

B1. Giao diện (Frontend Trigger): Sinh viên nhấn nút “Tham gia”. Frontend hiển thị trạng thái loading (disable nút để tránh click đúp) và gửi một request POST đến endpoint `/api/registrations` kèm theo `workshop_id`.

B2. Xác thực & Trích xuất định danh (Authentication): Backend nhận request. Middleware sẽ giải mã JWT Token nằm trong Header. Nếu hợp lệ, hệ thống trích xuất `user_id` và kiểm tra quyền (`role === 'STUDENT'`). Việc lấy `user_id` từ Token đảm bảo tính bảo mật, ngăn chặn việc user A cố tình gửi payload để đăng ký hộ/phá hoại user B.

B3. Kiểm soát tải (Rate Limiting): Trước khi chạm vào logic Database, request phải đi qua bộ lọc của Redis. Hệ thống đếm số lượng request của `user_id` (hoặc IP) trong 1 phút qua. Nếu dưới ngưỡng (ví dụ `< 30 req/phút`), request được cho phép đi tiếp.

B4. Thiết lập Khóa phân tán (Distributed Lock): Để chống tranh chấp (Race Condition), Backend yêu cầu Redis cấp một khóa độc quyền (Lock) cho `workshop_id` này (Ví dụ: key là `lock:workshop:123`).

- Chỉ request nào lấy được Lock mới được đi tiếp xuống Database.
- Các request đến cùng lúc cho cùng workshop này sẽ phải đứng chờ (sleep/retry) trong vòng vài trăm mili-giây cho đến khi Lock được mở, hoặc bị timeout nếu chờ quá lâu.

B5. Kiểm tra điều kiện (Validation): Request lấy được Lock sẽ mở một Database Transaction (phiên giao dịch) trên PostgreSQL và thực hiện các kiểm tra:

- Truy vấn xem bản ghi user_id và workshop_id này đã tồn tại trong bảng Registrations chưa (Chống đăng ký trùng).
- Truy vấn available_slots của workshop (sử dụng lệnh SELECT ... FOR UPDATE để khóa cứng dòng dữ liệu này ở mức cơ sở dữ liệu, đề phòng Redis bị lỗi). Phải đảm bảo available_slots > 0.

B6. Thực thi Giao dịch (Execute Transaction): Backend trừ số lượng chỗ: UPDATE Workshops SET available_slots = available_slots - 1.

- Khởi tạo chuỗi mã hóa (Hash) duy nhất bằng UUID v4 để làm vé QR.
- Tạo bản ghi mới trong bảng Registrations. Trạng thái thanh toán (payment_status) sẽ tự động gán là PAID nếu workshop miễn phí, hoặc PENDING nếu có thu phí.
- Commit Transaction lưu dữ liệu vĩnh viễn vào ổ cứng.

B7. Giải phóng tài nguyên (Release Lock): Lập tức xóa Lock trong Redis (DEL lock:workshop:123) để nhường đường cho request của các sinh viên khác đang chờ.

B8. Xử lý nền (Background Job): (Dành cho workshop miễn phí) Backend đẩy một thông điệp chứa registration_id vào Redis Message Queue. Một Worker chạy ngầm sẽ gấp thông điệp này ra và tiến hành gọi API gửi email nhà vé QR cho sinh viên.

B9. Phản hồi (Response): Backend trả về HTTP Status 200 OK (hoặc 201 Created). Frontend tắt loading và chuyển hướng sinh viên sang trang “Vé của tôi” (nếu miễn phí) hoặc trang “Thanh toán” (nếu có phí).

B10. Khởi tạo bộ đếm thời gian giữ chỗ (Hold Timeout - Dành cho vé có phí): Nếu là workshop có phí, Backend sẽ lên lịch một tác vụ ngầm (Delayed Job bằng Redis BullMQ) hẹn giờ đúng **15 phút** sau sẽ chạy. Nếu sau 15 phút mà vé vẫn ở trạng thái PENDING, hệ thống sẽ tự động hủy vé (status = CANCELLED) và hoàn trả lại số lượng chỗ (UPDATE Workshops SET available_slots = available_slots + 1)

3. Kịch bản lỗi (Edge Cases & Exception Handling)

3.1. Sinh viên bấm nút quá nhanh hoặc dùng Tool Spam (Tải đột biến)

- **Trigger:** Hệ thống ghi nhận số lượng request từ một user vượt mức cấu hình của thuật toán Token Bucket trên Redis (vd: > 30 req/phút).
- **Xử lý:** Chặn request ngay ở Gateway, không thực hiện truy vấn DB. Trả về mã lỗi 429 Too Many Requests. Frontend hiển thị thông báo toast: “Hệ thống đang quá tải, vui lòng thao tác chậm lại.”

3.2. Đăng ký trùng lặp (Duplicate Registration)

- **Trigger:** Mạng chậm khiến sinh viên nhấn nút đăng ký 2 lần, hoặc cố tình gửi API bằng Postman trong khi đã có vé của sự kiện này.
- **Xử lý:** Bước Validation ở tầng Database (hoặc ràng buộc Unique Constraint giữa user_id và workshop_id) sẽ phát hiện trùng lặp. Giao dịch bị hủy (Rollback), nhả Lock Redis. Trả về mã lỗi 409 Conflict. Frontend báo lỗi: “Bạn đã đăng ký tham gia sự kiện này rồi.”

3.3. Hết chỗ ngay khoảnh khắc đăng ký (Overbooking Attempt / Race Condition)

- **Trigger:** Chỉ còn 1 slot, nhưng Sinh viên A và B bấm cùng lúc. A lấy được Redis Lock trước. B đứng chờ. Khi A xử lý xong, số slot về 0. Lúc này B mới lấy được Lock và đi vào truy vấn DB.
- **Xử lý:** Ở bước kiểm tra điều kiện (Bước 5), DB xác nhận available_slots == 0. Giao dịch của B bị hủy (Rollback). Nhả Lock. Trả về mã lỗi 400 Bad Request hoặc 409 Conflict. Frontend báo: “Rất tiếc, sự kiện vừa hết chỗ. Vui lòng chọn sự kiện khác.”

3.4. Quá thời gian chờ Khóa (Lock Timeout)

- **Trigger:** Có quá nhiều người (ví dụ 1.000 người) cùng tranh 1 workshop. Hàng đợi chờ lấy Redis Lock quá dài, request của sinh viên phải đợi quá 3 giây mà chưa tới lượt.
- **Xử lý:** Để tránh việc kết nối HTTP bị treo (timeout vòng đời request), hàm lấy Lock sẽ tự động bỏ cuộc (Fail-fast) sau 3 giây chờ đợi. Trả về mã lỗi 503 Service Unavailable. Frontend thông báo: “Hệ thống đang bận do lượng người truy cập lớn, vui lòng thử lại.”

3.5. Sự cố đứt gãy Redis (Redis Crash)

- **Trigger:** Máy chủ chứa Redis bị sập. Hệ thống không thể dùng Redis để đếm Rate Limit hay phân phát Distributed Lock.
- **Xử lý:** Áp dụng Graceful Degradation. Backend bắt được lỗi kết nối Redis, tự động bỏ qua bước kiểm tra Rate Limit và bỏ qua bước lấy Distributed Lock. Yêu cầu đăng ký đầy thẳng xuống PostgreSQL. Lúc

này, cơ chế **Row-level Lock** (SELECT ... FOR UPDATE) của bản thân PostgreSQL sẽ gánh vác trách nhiệm chống Overbooking. Hiệu năng có thể giảm (database xử lý chậm hơn), nhưng tính toàn vẹn dữ liệu vẫn đạt 100%.

3.6. Lỗi kẹt giao dịch Database (Deadlock / DB Timeout)

- **Trigger:** PostgreSQL bị nghẽn CPU do tải quá nặng, dẫn đến các Transaction kéo dài quá giới hạn thời gian cho phép.
- **Xử lý:** Database tự động ném ra lỗi Deadlock/Timeout. Backend Catch lỗi này, thực hiện thao tác Rollback để đảm bảo dữ liệu không bị rác. Trả về 500 Internal Server Error. Hệ thống giám sát (Monitoring) tự động gửi cảnh báo cho Admin.

3.7. Vé bị hủy do quá hạn thanh toán (Payment Timeout)

- **Trigger:** Sau 15 phút kể từ lúc giữ chỗ thành công, hệ thống không nhận được xác nhận thanh toán thành công từ Mock Payment Gateway.
- **Xử lý:** Delayed Job kích hoạt. Chuyển payment_status của bản ghi Registrations thành CANCELLED. Trả lại 1 available_slots cho Workshop. Gửi email thông báo cho sinh viên: “Giao dịch giữ chỗ của bạn đã bị hủy do quá thời gian thanh toán.”

4. Ràng buộc (Constraints)

- Tính nhất quán (Consistency): Bắt buộc phải áp dụng tính chất ACID của hệ quản trị cơ sở dữ liệu quan hệ. Tuyệt đối không cho phép 2 sinh viên cùng nhận được thông báo đăng ký thành công khi chỉ còn 1 slot cuối cùng.
- Hiệu năng (Performance): Thời gian phản hồi của API đăng ký không được vượt quá 500ms ở điều kiện bình thường và 1500ms ở điều kiện tải đỉnh (peak load 7.200 req/3 phút).
- Bảo mật: Backend tuyệt đối không lấy user_id từ req.body do client gửi lên (để ngăn chặn giả mạo/đăng ký hộ). user_id bắt buộc phải được giải mã (decode) trực tiếp từ JWT Token ở tầng Middleware.

5. Tiêu chí chấp nhận (Acceptance Criteria)

- **Test Case 1:** Gửi 100 HTTP request đồng thời (Concurrent requests) bằng JMeter/K6 vào một workshop chỉ còn đúng 1 slot trống. Hệ thống chỉ cho phép đúng 1 request trả về status `200 OK`, 99 request còn lại phải trả về `409 Conflict`.

- **Test Case 2:** Sinh viên cố tình gửi 2 request đăng ký cùng một `workshop\_id` cho chính mình. Request thứ 2 phải bị từ chối với thông báo “Bạn đã đăng ký sự kiện này”.
- **Test Case 3:** User đăng nhập với tài khoản có role `ORGANIZER` gọi API đăng ký. Hệ thống trả về `403 Forbidden`.
- **Test Case 4:** Sinh viên A đăng ký thành công workshop có phí (còn đúng 1 slot cuối), nhưng không thanh toán. Sinh viên B vào sau thấy báo “Hết chỗ”. Đúng 15 phút sau, hệ thống hủy vé của A. Sinh viên B f5 lại trang, thấy còn 1 slot trống và có thể đăng ký thành công.